CHAPTER

Application Case Study: Molecular Visualization and Analysis

9

With special contributions from John Stone

CHAPTER CONTENTS

9.1 Application Background 174
9.2 A Simple Kernel Implementation 176
9.3 Instruction Execution Efficiency 180
9.4 Memory Coalescing 182
9.5 Additional Performance Comparisons 185
9.6 Using Multiple GPUs 187
9.7 Exercises 188
References and Further Reading 189

INTRODUCTION

The previous case study illustrated the process of selecting the appropriate level of a loop nest for parallel execution, the use of constant memory for magnifying the memory bandwidth for read-only data, the use of registers to reduce the consumption of memory bandwidth, and the use of special hardware functional units to accelerate trigonometry functions. In this case study, we use an application based on regular grid data structures to illustrate the use of additional practical techniques that achieve global memory access coalescing and improved computation throughput. We present a series of implementations of an electrostatic potential map calculation kernel, with each version improving upon the previous one. Each version adopts one or more practical techniques. The computation pattern of this application is one of the best matches for graphics processing unit (GPU) computing. This case study shows that the effective use of these practical techniques can significantly improve the execution throughput and is critical for the application to achieve its potential performance.

9.1 APPLICATION BACKGROUND

This case study is based on Visual Molecular Dynamics (VMD) [Humphrey 1996], a popular software system designed for displaying, animating, and analyzing biomolecular systems. As of 2009, VMD had more than 10,000 registered users. Although it has strong built-in support for analyzing biomolecular systems, such as calculating electrostatic potential values at spatial grid points of a molecular system, it has also been a popular tool for displaying other large datasets such as sequencing data, quantum chemistry simulation data, and volumetric data due to its versatility and user extensibility.

Although VMD is designed to run on a diverse range of hardware—laptops, desktops, clusters, and supercomputers—most users utilize VMD as a desktop science application for interactive three-dimensional (3D) visualization and analysis. For computation that runs too long for interactive use, VMD can also be used in a batch mode to render movies for later use. A motivation for using GPU acceleration in VMD is to make slow, batch-mode jobs fast enough for interactive use. This can drastically improve the productivity of scientific investigations. With CUDA™-enabled GPUs widely available on desktop PCs, such acceleration can have a broad impact on the VMD user community. To date, multiple aspects of VMD have been accelerated with CUDA, including electrostatic potential calculation, ion placement, molecular orbital calculation and display, and imaging of gas migration pathways in proteins.

The particular calculation used in this case study is that of electrostatic potential maps in a grid space. This calculation is often used in placement of ions into a structure for molecular dynamics simulation. Figure 9.1 shows the placement of ions into a protein structure in preparation for molecular dynamics simulation. In this application, the electrostatic potential map is used to identify spatial locations where ions (dark dots) can fit in according to physical laws. The function can also be used to calculate time-averaged potentials during molecular dynamics simulation, which is useful for the simulation process as well as the visualization and analysis of simulation results.

Among the several methods for calculating electrostatic potential maps, direct Coulomb summation (DCS) is a highly accurate method that is particularly suitable for GPUs [Stone 2007]. The DCS method calculates the electrostatic potential value of each grid point as the sum of contributions from all atoms in the system. This is illustrated in Figure 9.2. The contribution of atom i to lattice point j is the charge of that atom divided by the distance from lattice point j to atom i. Because this must be done for all grid

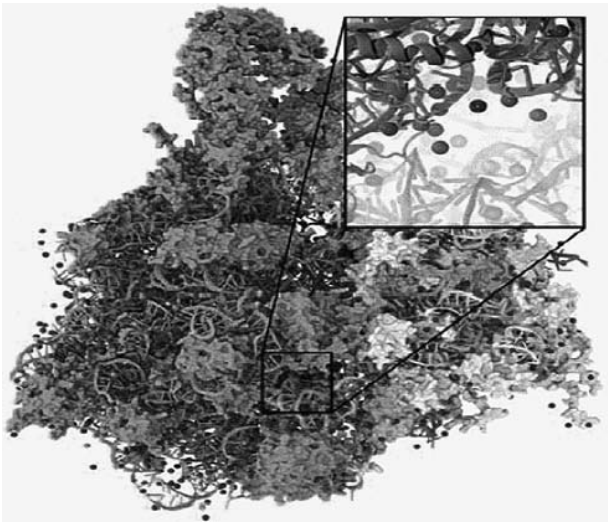


FIGURE 9.1

Electrostatic potential map used to build stable structures for molecular dynamics simulation.

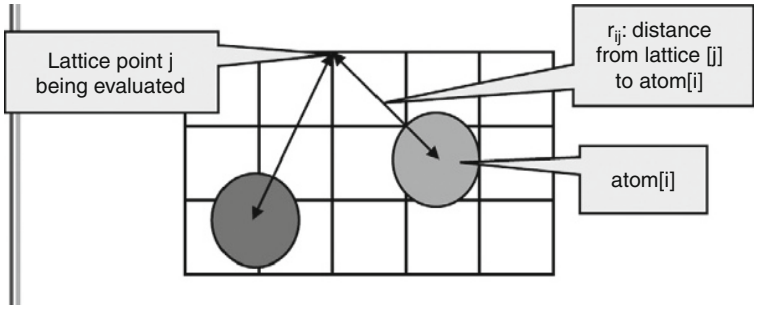


FIGURE 9.2

The contribution of `atom[i]` to the electrostatic potential at lattice point *j* (`potential[j]`) is `atom[i].charge/rij`. In the direct Coulomb summation (DCS) method, the total potential at lattice point *j* is the sum of contributions from all atoms in the system.

points and all atoms, the number of calculations is proportional to the product of the total number of atoms in the system and the total number of grid points. For a realistic molecular system, this product can be very large; therefore, the calculation of the electrostatic potential map has been traditionally done as a batch job in VMD.

9.2 A SIMPLE KERNEL IMPLEMENTATION

Figure 9.3 shows the base C code of the DCS code. The function is written to process a two-dimensional (2D) slice of a three-dimensional grid. The function will be called repeatedly for all the slices of the modeled space. The structure of the function is quite simple, with three levels of `for` loops. The outer two levels iterate over the *y*-dimension and the *x*-dimension of the grid-point space. For each grid point, the innermost `for` loop iterates over all atoms, calculating the contribution of electrostatic potential energy from all atoms to the grid point. Note that each atom is represented by four consecutive elements of the `atoms[]` array. The first three elements store the *x*, *y*, and *z* coordinates of the atom and the fourth element the electrical charge of the atom. At the end of the innermost loop, the accumulated value of the grid point is written out to the grid data structure. The outer loops then iterate and take the execution to the next grid point.

Note that the DCS function in Figure 9.3 calculates the *x* and *y* coordinates of each grid point on the fly by multiplying the grid-point index values by the spacing between grid points. This is a uniform grid method, where all grid points are spaced at the same distance in all three dimensions. The function does take advantage of the fact that all the grid

```
void cenergy(float*energygrid, dim3 grid, float gridspacing, float z, const float *atoms,
            int numatoms) {
 int i,j,n;
 int atomarrdim = numatoms * 4;
 for (j=0; j<grid.y; j++) {
  float y = gridspacing * (float) j:
  for (i=0; i<grid.x; i++) {
   float x = gridspacing * (float) i;
   float energy = 0.0f;
   for (n=0; n<atomarrdim; n+=4) {  //calculate potential contribution of each atom
    float dx = x - atoms[n];
    float dy = y - atoms[n+1];
    float dz = z - atoms[n+2];
    energy += atoms[n+3]/sqrtf(dx*dx + dy*dy + dz*dz);
   }
   energygrid[grid.x*grid.y*k + grid.x*j + i] = energy;
  }
 }
```

FIGURE 9.3

Base Coulomb potential calculation code for a two-dimensional slice of the three-dimensional grid.

points in the same slice have the same z coordinate. This value is precalculated by the caller of the function and passed in as a function parameter (z).

Based on what we learned from the magnetic resonance imaging (MRI) case study, two attributes of the DCS method should be apparent. First, the computation is massively parallel; the computation of electrostatic potential for each grid point is independent of that of other grid points. There are two alternative approaches to organizing parallel execution. In the first option, we can use each thread to calculate the contribution of one atom to all grid points. This would be a poor choice, as each thread would be writing to all grid points, requiring extensive use of atomic memory operations to coordinate the updates done by different threads to each grid point. The second option uses each thread to calculate the accumulated contributions of all atoms to one grid point. This is a preferred approach, as each thread will be writing into its own grid point and it is not necessary to use atomic operations.

We will form a two-dimensional thread grid that matches the two-dimensional energy grid-point organization. In order to do so, we need to modify the two outer loops into perfectly nested loops so we can use each thread to execute one iteration of the two-level loop. We can either perform a loop fission (introduced in Section 8.3) or move the calculation of the y coordinate into the inner loop. The former would require us to create a new array to hold all y values and result in two kernels communicating data through global memory. The latter increases the number of times that the y coordinate will be calculated. In this case, we choose to perform the latter because there is only on calculation that can be easily accommodated into the inner loop without significant increase in execution time of the inner loop. The former would have added a kernel launch overhead for a kernel where threads do very little work. The selected transformation allows all i and j iterations to be executed in parallel. This is a tradeoff between the amount of calculation done and the level of parallelism achieved.

The second experience that we can apply from the MRI case study is that the electrical charge of every atom will be read by all threads. This is because every atom contributes to every grid point in the DCS method. Furthermore, the values of the atomic electrical charges and their positions are not modified during the computation. This means that the atomic charge and position values (the contents of the `atoms[]` array) can be efficiently stored in the constant memory (in the GPU box in Figure 9.4). Because there can be more than 64 kB of atom charges in the input, we need to partition them into 64-kB chunks in the same way that we did for the MRI case study.

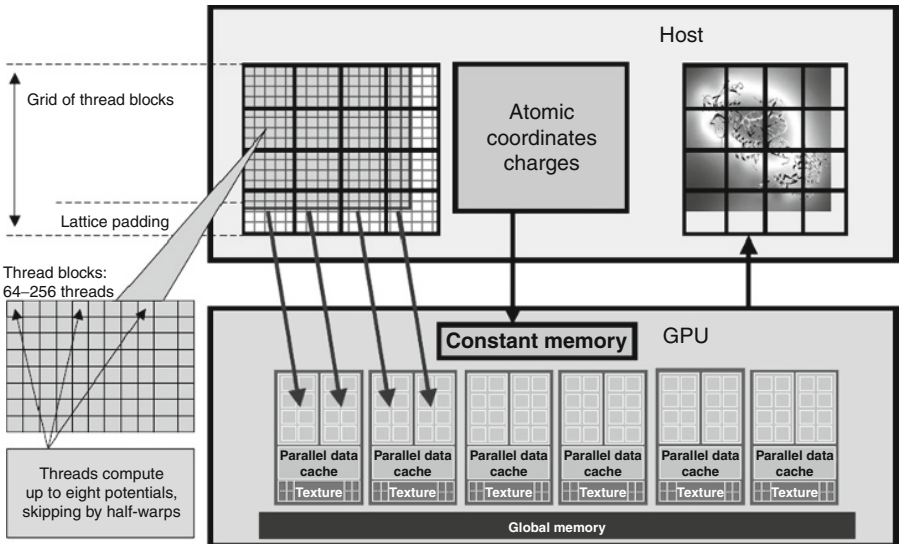


FIGURE 9.4

Overview of the DSC kernel design.

Figure 9.4 shows an overview of the DCS kernel design. The host program ("Host" box) inputs and maintains the atomic charges and their coordinates in the system memory. It also maintains the grid-point data structure in the system memory (left side of the "Host" box). The DCS kernel is designed to process a 2D slice of the energy grid-point structure (not to be confused with thread grids). The grid on the right side of the "Host" box shows an example of a 2D slice. For each 2D slice, the central processing unit (CPU) transfers its grid data to the device global memory. The atom information is divided into chunks to fit into the constant memory. For each chunk of the atom information, the CPU transfers the chunk into the device constant memory, invokes the DCS kernel to calculate the contribution of the current chunk to the current slice, and prepares to transfer the next chunk. After all chunks of the atom information have been processed for the current slice, the slice is transferred back to update the grid-point data structure in the CPU system memory. The system then moves on to the next slice.

Within each kernel invocation, the thread blocks are organized to calculate the electrostatic potential of tiles of the grid structure. In the simplest kernel, each thread calculates the value at one grid point. In more sophisticated kernels, each thread calculates multiple grid points and exploits the

redundancy between the calculations of the grid points to improve execution speed. This is illustrated in the left side of the figure, labeled as "Thread blocks."

Figure 9.5 shows the resulting CUDA kernel code. We omitted some of the declarations. As in the case the MRI case study, the `atominfo[]` array is the device code counter part of the `atoms[]` array in the host base code of Figure 9.3. The host code also needs to divide up the atom information into chunks that fit into the constant memory for each kernel invocation. This means that the kernel will be invoked multiple times when there are multiple chunks' worth of atoms. Because this is similar to the MRI case study, we will not show the details.

The outer two levels of the loop in Figure 9.3 have been removed from the kernel code and are replaced by the execution configuration parameters in the kernel invocation. This is also similar to one of the steps we took the MRI case study, so we will not show the kernel invocation but leave it as an exercise for the reader. The rest of the kernel code is straightforward and corresponds directly to the original loop body of the innermost loop.

One particular aspect of the kernel is somewhat subtle and worth mentioning. The kernel code calculates the contribution of a chunk of atoms to a grid point. The grid point must be preserved in the global memory and updated by each kernel invocation. This means that the kernel needs to read the current grid point value, add the contributions by the current

```
. . .
 float curenergy = energygrid[outaddr];
 float coorx = gridspacing * xindex;
 float coory = gridspacing * yindex;
 int atomid;
 float energyval=0.0f;
  for (atomid=0; atomid<numatoms; atomid++) {
   float dx = coorx - atominfo[atomid].x;
   float dy = coory - atominfo[atomid].y;
   energyval += atominfo[atomid].w*
                    rsqrtf(dx*dx + dy*dy + atominfo[atomid].z);
 }
   energygrid[outaddr] = curenergy + energyval;
```

Start global memory reads early. Kernel hides some of its own latency.

Only dependency on global memory read is at the end of the kernel...

FIGURE 9.5

DCS Kernel Version 1.

chunk of atoms, and write the updated value to global memory. The code attempts to hide the global memory latency by loading the grid value at the beginning of the kernel and using it at the end of the kernel. This helps to reduce the number of warps needed by the streaming multiprocessor (SM) scheduler to hide the global memory latency.

The performance of the kernel in Figure 9.5 is quite good, having been measured at 186 gigaflops on an NVIDIA® G80. In terms of application-level performance, the implementation can process 18.6 billion atom evaluations per second. A quick glance over the code shows that each thread does nine floating-point operations for every four memory elements accessed. On the surface, this is not a very good ratio. We need a ratio of at least 8 to avoid global memory congestion; however, all four memory accesses are done to an `atominfo[]` array. These `atominfo[]` array elements for each atom are cached in a hardware cache memory in each SM and are broadcast to a large number of threads. A calculation similar to that in the MRI case study shows that the massive reuse of memory elements across threads makes the constant cache extremely effective, boosting the effective ratio of floating operations per global memory access much higher than 10:1. As a result, global memory bandwidth is not a limiting factor for this kernel.

9.3 INSTRUCTION EXECUTION EFFICIENCY

Although the kernel in Figure 9.5 avoids global memory bottleneck through constant caching, it still must execute four constant memory access instructions for every nine floating-point operations performed. These memory access instructions consume hardware resources that could be otherwise used to increase the execution throughput of floating point instructions. This section shows that we can fuse several threads together so the `atominfo[]` data can be fetched once from the constant memory, stored into registers, and used for multiple grid points. This idea is illustrated in Figure 9.6.

Furthermore, all grid points along the same row have the same y coordinate; therefore, the difference between the y coordinate of an atom and the y coordinate of any grid point along a row has the same value. In the DCS Kernel Version 1 in Figure 9.5, this calculation is redundantly done by all threads for all grid points in a row when calculating the distance between the atom and the grid points. We can eliminate this redundancy and improve the execution efficiency.

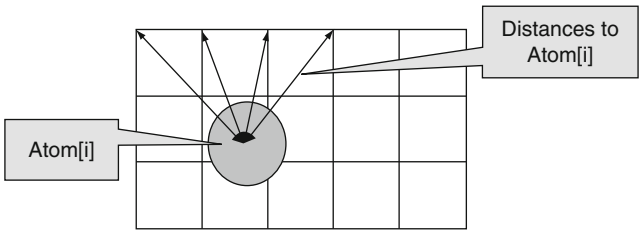


FIGURE 9.6

Reusing information among multiple grid points.

The idea is to have each thread to calculate the electrostatic potential for multiple grid points. In the kernel shown in Figure 9.7, each thread calculates four grid points. For each atom, the code calculates `dy`, the difference of the *y* coordinate in line 2. It then calculates the expression `dy*dy` plus the precalculated `dz*dz` information and saves it to the auto variable `dysqpdzsq`, which is assigned to a register by default. This value is the same for all four grid points; therefore, the calculation of `energyvalx1` through `energyvalx4` can just use the value stored in the register. Furthermore, the electrical charge information is also accessed from constant memory and stored in the automatic variable `charge`. Similarly, the *x* coordinate of the atom is also read from constant memory into auto variable `x`. Altogether, this kernel eliminates three accesses to constant memory for `atominfo[atomid].y`, three accesses to constant memory for `atominfo[atomid].x`, three accesses to constant memory for `atominfo[atomid].w`,

```
...for (atomid=0; atomid<numatoms; atomid++) {
    float dy = coory - atominfo[atomid].y;
    float dysqpdzsq = (dy * dy) + atominfo[atomid].z;
    float x = atominfo[atomid].x;
    float dx1 = coorx1 - x;
    float dx2 = coorx2 - x;
    float dx3 = coorx3 - x;
    float dx4 = coorx4 - x;
    float charge = atominfo[atomid].w;
    energyvalx1 += charge * rsqrtf(dx1*dx1 + dysqpdzsq);
    energyvalx2 += charge * rsqrtf(dx2*dx2 + dysqpdzsq);
    energyvalx3 += charge * rsqrtf(dx3*dx3 + dysqpdzsq);
    energyvalx4 += charge * rsqrtf(dx4*dx4 + dysqpdzsq);
    }
```

Compared to non-unrolled kernel: memory loads are decreased by 4x, and FLOPS per evaluation are reduced, but register use is increased...

FIGURE 9.7

DCS Kernel Version 2.

three floating-point subtraction operations, five floating-point multiply operations, and nine floating-point add operations when processing an atom for four grid points. A quick inspection of the kernel code in Figure 9.7 should show that each iteration of the loop performs four constant memory accesses, five floating-point subtractions, nine floating-point additions, and five floating-point multiplications for four grid points.

The reader should also verify that the version of DCS kernel in Figure 9.5 performs 16 constant memory accesses, 8 floating-point subtractions, 12 floating-point additions, and 12 floating-point multiplications, a total of 48 operations for the same four grid points. Going from Figure 9.5 to Figure 9.7, there is a dramatic reduction from 48 operations down to 25 operations. This is translated into an increased execution speed from 186 gigaflops to 259 gigaflops. In terms of application-level throughput, the performance increases from 18.6 billion atom evaluations per second to 33.4 billion atom evaluations per second. The reason why the application-level performance improvement is higher than the floating-point operations per second (FLOPS) improvement is that some of the floating-point operations have been eliminated.

The cost of the optimization is that more registers are used by each thread. This reduces the number of threads that can be assigned to each SM. However, as the results show, this is a good tradeoff providing an excellent performance improvement.

9.4 MEMORY COALESCING

The performance of the DCS Kernel Version 2 shown in Figure 9.7 is quite high, but a quick profiling run reveals that the threads perform memory writes inefficiently. As shown in Figures 9.6 and 9.7, each thread calculates four neighboring grid points. This seems to be a reasonable choice; however, as we illustrate in Figure 9.8, the access pattern of threads will result in uncoalesced global memory writes.

We can identify two problems that are causing the uncoalesced writes in DCS Kernel Version 2. First, each thread calculates four adjacent neighboring grid points; thus, for each statement that access the `energygrid[]` array, the threads in a warp are not accessing adjacent locations. Note that two adjacent threads access memory locations that are three elements apart. Thus, the 16 locations to be written by all the threads in warp write are spread out, with three elements in between the loaded/written locations. This problem can be solved by assigning adjacent grid points to adjacent

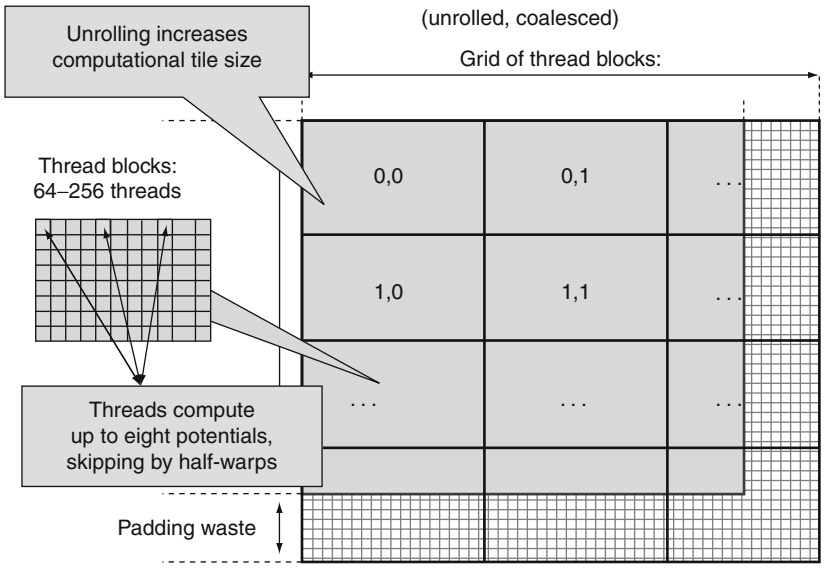


FIGURE 9.8

Organizing threads and memory layout for coalesced writes.

threads in each half-warp. Assuming that we still want to have each thread calculate 4 grid points, we first assign 16 consecutive grid points to the 16 threads in a half-warp. We then assign the next 16 consecutive grid points to the same 16 threads. We repeat the assignment until each thread has the number of grid points desired. This assignment is illustrated in Figure 9.8. With some experimentation, the best number of grid points per thread turns out to be 8 for the G80.

The kernel code with warp-aware assignment of grid points to threads is shown in Figure 9.9. Note that the *x* coordinates used to calculate the distances are offset by the variable `gridspacing_coalescing`, which is the original grid spacing times the constant `BLOCKSIZEX` (in this case, 16). This reflects the fact that the *x* coordinates of the 8 grid points are 16 grid points away from each other. Also, after the end of the loop, memory writes to the `energygrid[]` array are indexed by `outaddr`, `outaddr+BLOCKSIZEX`, . . ., `outaddr+7*BLOCKSIZEX`. Each of these indices is one `BLOCKSIZEX` (16) away from the previous one. The detailed thread block organization for this kernel is left as an exercise. The reader should keep in mind that by setting the *x*-dimension size of the thread block to be equal to the half-warp size (16), we can simplify the indexing in the kernel.

```
...float coory = gridspacing * yindex;
   float coorx = gridspacing * xindex;
   float gridspacing_coalesce = gridspacing * BLOCKSIZEX;
   int atomid;
   for (atomid=0; atomid<numatoms; atomid++) {
     float dy = coory - atominfo[atomid].y;
     float dyz2 = (dy * dy) + atominfo[atomid].z;
     float dx1 = coorx – atominfo[atomid].x;
[...]
     float dx8 = dx7 + gridspacing_coalesce;
     energyvalx1 += atominfo[atomid].w * rsqrtf(dx1*dx1 + dyz2);
[...]
     energyvalx8 += atominfo[atomid].w * rsqrtf(dx8*dx8 + dyz2);
   }
   energygrid[outaddr                        ] += energyvalx1;
[...]
   energygrid[outaddr+7*BLOCKSIZEX] += energyvalx7;
```

Points spaced for memory coalescing

Reuse partial distance components dy^2 + dz^2

Global memory ops occur only at the end of the kernel, decreases register use

FIGURE 9.9

DCS Kernel Version 3.

The other cause of uncoalesced memory writes is the layout of the `energygrid[]` array, which is a three-dimensional array. If the x-dimension of the array is not a multiple of half-warp size, the beginning location of the second row, as well as the beginning locations of the subsequent rows, will no longer be at the 16-word boundaries. This means that the half-warp accesses will not be coalesced, even though they write to consecutive locations. This problem can be corrected by padding each row with additional elements so the total length of the x-dimension is a multiple of 16. This can require adding up to 15 elements, or 60 bytes to each row, as shown in Figure 9.8. With the kernel of Figure 9.9, the number of elements in the x-dimension needs to be a multiple of $8 \times 16 = 128$. This is because each thread actually writes eight elements in each iteration. Thus, one may need to add up to 127 elements, or 1016 bytes, to each row.

Furthermore, there is a potential problem with the last row of thread blocks. Because the grid array may not have enough rows, some of the threads may end up writing outside the grid data structure. The grid data structure is a 3D array, so these threads will write into the next slice of grid points. One potential solution is to add a test in the kernel and avoid writing the array elements that are out of the known y-dimension size; however, this would have added a number of overhead instructions and incurred

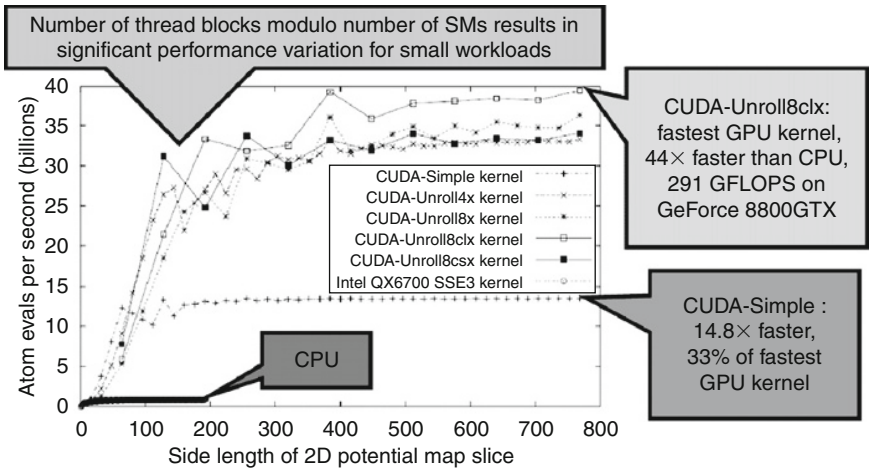
control divergence. A more desirable solution is to pad the *y*-dimension of the grid structure so it contains a multiple of tiles covered by thread blocks. This is shown in Figure 9.8 as the bottom padding in the grid structure. In general, one may need to add up to 15 rows due to this padding.

The cost of padding can be substantial for smaller grid structures. For example, if the energy grid has 100×100 grid points in each 2D slice, it would be padded into a 128×112 slice. The total number of grid points would increase from 10,000 to 14,336, or a 43% overhead. If we had to pad the entire 3D structure, the grid points would increase from $100 \times 100 \times 100 = 1{,}000{,}000$ to $128 \times 112 \times 112 = 1{,}605{,}632$, or a 60% overhead! This is part of the reason why we calculate the energy grids in 2D slices and use the host code to iterate over these 2D slices. Writing a single kernel to process the entire 3D structure would incur a lot more extra overhead. This type of tradeoff appears frequently in simulation models, differential equation solvers, and video processing applications.

The DCS Kernel Version 3 kernel shown in Figure 9.9 achieves about 291 gigaflops, or 39.5 billion atom evaluations per second. The measured speed of this kernel also has a slight enhancement from moving the read access to the `energygrid[]` array from the beginning of the kernel to the end of the kernel. The contributions to the grid points are first calculated in the loop. The code loads the original grid-point data after the loop, adds the contribution to them, and writes the updated values back. Although this movement exposes more of the global memory latency to each thread, it saves the consumption of eight registers. Because the kernel is already using many registers to hold the atom data and the distances, such saving in number of registers used relieves a critical bottleneck for the kernel. This allows more thread blocks to be assigned to each SM and achieves an overall performance improvement.

9.5 ADDITIONAL PERFORMANCE COMPARISONS

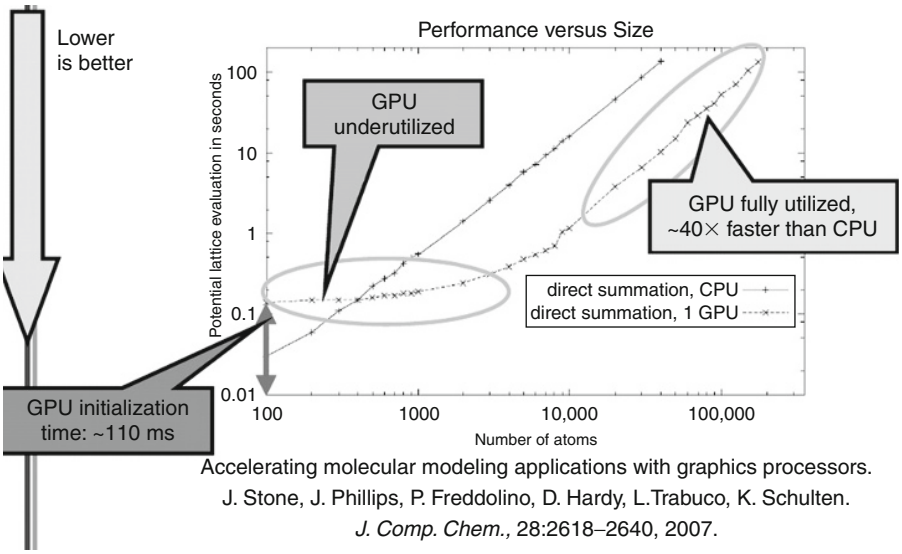
Figure 9.10 shows a summary of the performance comparison between the various DCS kernel implementations and how they compare with an optimized single-core CPU execution. One important observation is that the relative merit of the kernels varies with grid dimension lengths; however, DCS Kernel Version 3 (CUDA-Unroll8clx) from Figure 9.9 performs consistently better than all others once the grid dimension length is larger than 300. The notation Unroll8clx indicates that the kernel assigns 8 grid points to each thread and uses padding and strided grid point assignment to threads to ensure coalescing.



GPU computing. J. Owens, M. Houston, D. Luebke, S. Green, J. Stone, J. Phillips. *Proceedings of the IEEE*, 96:879-899, 2008.

FIGURE 9.10

Performance comparison of various DCS kernel versions.



Accelerating molecular modeling applications with graphics processors. J. Stone, J. Phillips, P. Freddolino, D. Hardy, L.Trabuco, K. Schulten. *J. Comp. Chem.,* 28:2618–2640, 2007.

FIGURE 9.11

CPU versus CPU/GPU comparison.

A detailed comparison of the CPU performance and the CPU/GPU joint performance reveals a commonly observed tradeoff. Figure 9.11 shows a plot of the execution time of a medium-sized grid system for varying numbers of atoms to be evaluated. For 400 atoms or fewer, the CPU performs better. This is because the GPU has a fixed initialization overhead of 110 ms, regardless of the number of atoms to be evaluated. Also, for a small number of atoms, the GPU is underutilized, thus the curve of the GPU execution time is quite flat, between 100 and 1000 atoms.

The plot in Figure 9.11 reinforces a commonly held principle that GPUs perform better for large amounts of data. Once the number of atoms reaches 10,000, the GPU is fully utilized. The slopes of the CPU and the CPU/GPU execution times become virtually identical, with the CPU/GPU execution being consistently 44× faster than the CPU execution for all input sizes.

9.6 USING MULTIPLE GPUs

In Section 9.2, we gave an overview of the DCS kernel design. Recall that for each slice of the energy grid structure, the CPU first transfers the slice to the device memory and then streams the atom information through the device constant memory. This design provides a very straightforward path to using multiple GPUs to further improve the execution speed of the kernel.

For a multi-GPU design, the CPU first launches multiple p-threads. The CUDA runtime requires that each GPU must be associated with a separate p-thread running on the CPU. The CPU then forms a task pool, each task being the processing of one energy grid slice. When one of the GPUs becomes available, its corresponding p-thread picks up a task (slice), transfers the slice to the GPU, and begins to stream the atom information through the device constant memory of that GPU. In this way, the GPUs collaboratively process the pool of slices, thus increasing the rate of processing the grid. Using three GeForce® 8800 GTX GPUs, we have measured 863 gigaflops, or 117 billion atom evaluations per second. This is a 131× speedup over a single CPU core. This was measured from a PC with three GPU cards, as shown in Figure 9.12. With such speed, experiments that used to take a month on a PC box now take a few hours. This has been shown to greatly improve the productivity of researchers in the area of molecular science and medicine.

Although the use of multiple GPUs for the DCS kernel is conceptually simple, the detailed mechanics for creating CPU p-threads, creating a task pool, and handling errors and exception conditions can be tedious and error

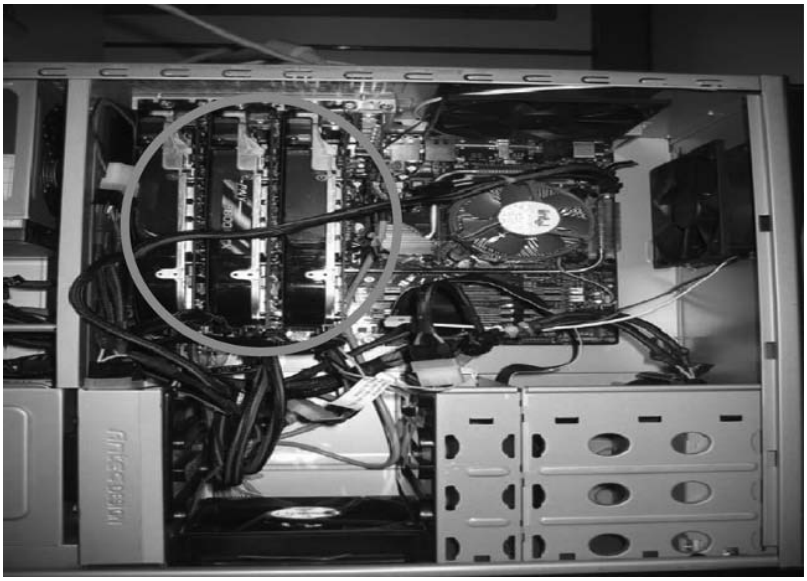


FIGURE 9.12

A PC with three GeForce® 8800 GTX GPUs at 700 W of total system power.

prone. This has motivated the creation of the WorkForce framework and runtime to simplify the use of multiple GPUs. Interested readers should download WorkForce for their multiple-GPU applications.

9.7 EXERCISES

9.1 Complete the implementation of the DCS kernel shown in Figure 9.5. Fill in all of the missing declarations. Develop a kernel launch statement with all of the execution configuration parameters.

9.2 Compare the number of operations (memory loads, floating-point arithmetic, branches) executed in each iteration of the kernel shown in Figure 9.7 compared to that in Figure 9.5. Keep in mind that each iteration of the former corresponds to four iterations of the latter.

9.3 Complete the implementation of the DCS kernel shown in Figure 9.9. Explain in your own words how the thread accesses are coalesced in this implementation.

9.4 For the memory padding in Figure 9.8 and the DCS kernel shown in Figure 9.9, show why one needs to pad up to 127 elements in the *x*-dimension but only up to 15 elements in the *y*-dimension.

9.5 Give two reasons for adding extra "padding" elements to arrays allocated in the GPU global memory, as shown in Figure 9.8.

9.6 Give two potential disadvantages associated with increasing the amount of work done in each CUDA thread, as shown in Section 9.3.

References and Further Reading

Humphrey, W., Dalke, A., & Schulten, K. (1996). VMD: Visual molecular dynamics. *Journal of Molecular Graphics*, *14*, 33–38.

Stone, J. E., Phillips, J. C., Freddolino, P. L., Hardy, D. J., Trabuco, L. G., & Schulten, K. (2007). Accelerating molecular modeling applications with graphics processors. *Journal of Computational Chemistry*, *28*, 2618–2640.

This page intentionally left blank